

PHYS2022: Physics from Evidence

Nuclear Fission Simulation

Taan Mudan

November 21, 2025

1 Introduction

This project implemented Monte Carlo simulations to model nuclear fission chain reactions and estimate the critical mass of uranium-235. Nuclear fission involves uranium atoms splitting after neutron absorption, releasing energy and producing additional neutrons. In small uranium volumes, most neutrons escape without causing further fissions, but above a critical size, a self-sustaining chain reaction can occur.

Computational models in both 1D and 3D were used to track neutrons from initial fissions and count secondary fission events. The 1D model served as a simplified starting point for understanding the basic physics, while the 3D extension provided more realistic predictions. The primary objective was to determine the critical dimensions where the average number of secondary fissions per initial fission equals 1.0 for a self-sustaining chain reaction.

5.1: 1D Basic Fission Model

The simulation begins with a simplified one-dimensional model to create a base to work on. A uranium bar of length $L = 0.1$ m is modelled using physical constants $a = 0.017$ m and $b = 0.21$ m, from which the characteristic diffusion distance $R = \sqrt{2ab} = 0.0845$ m can be found. This represents the typical distance neutrons travel before causing fission.

The code implements the Monte Carlo process by placing 100 initial fissions at random positions along the bar using $x_0 = L \times \text{random.random}()$. Each fission produces exactly 2 neutrons that diffuse left or right by distance R with equal probability, determined by the random direction check. Only neutrons that remain within $[0, L]$ after diffusion can cause secondary fissions, which are counted to measure chain reaction sustainability.

```
1 L = 0.1
2 a = 0.017
3 b = 0.21
4 R = np.sqrt(2 * a * b)
5 N_initial = 100
6 neutrons_per_fission = 2
7
8 secondary_count = 0
9 for i in range(N_initial):
10     x0 = L * random.random()
11     for j in range(neutrons_per_fission):
12         if random.random() > 0.5:
13             x_new = x0 + R
14         else:
15             x_new = x0 - R
16         if 0 <= x_new <= L:
17             secondary_count += 1
```

Multiple simulation runs gave secondary fissions values ranging from 25 to 39 from 100 initial fissions, giving an average range of 0.25 to 0.39 secondary fissions per initial fission. The results are consistently below 1 which confirms that the chain reaction would die out for this small length. The wide range of 14 fissions relative to the 100 initial fissions demonstrates the limitations of small sample sizes in this kind of simulation.

5.2: Improved 1D Model with Neutron Variation

The model was then enhanced by replacing the fixed 2 neutrons per fission with the `neutrons()` function, which generates random integers averaging 2.5 to more closely match reality.

```

1 def neutrons():
2     i = int(random.normal() + 3.0)
3     if (i < 0): return 0
4     else: return i
5
6 lengths = [0.1, 0.12, 0.14, 0.16, 0.18, 0.2]
7 N_variable = 5000

```

5.3: 1D Critical Length Determination

Testing across six different bar lengths from 0.10 m to 0.20 m was conducted with 5000 initial fissions per length, to attempt to find a value for L that gets close to the critical value.

```

1     for L in lengths:
2         secondary_total = 0
3         for i in range(N_variable):
4             x0 = L * random.random()
5             n_neutrons = neutrons()
6             for j in range(n_neutrons):
7                 x_new = x0 + R if random.random() > 0.5 else x0 - R
8                 if 0 <= x_new <= L:
9                     secondary_total += 1
10         avg_per_fission = secondary_total / N_variable

```

The value of L that gave a results closest to 1 average secondary fission was $L = 0.14$ m, where averages typically range from 0.98 to 1.02.

A higher resolution search then located the critical length using 201 equally spaced points between 0.14 m and 0.16 m, to determine a more precise critical value of L where the average secondary fissions equals exactly 1.0.

```

1 Ls = np.linspace(0.14, 0.16, 201)
2 avg = []
3
4 for L in Ls:
5     secondary_total = 0
6     for i in range(N_variable):
7         x0 = L * random.random()
8         n = neutrons()
9         for j in range(n):
10            x_new = x0 + R if random.random() > 0.5 else x0 - R
11            if 0 <= x_new <= L:
12                secondary_total += 1
13         avg.append(secondary_total / N_variable)
14

```

```

15 min_diff = 1000
16 best_index = 0
17 for i in range(len(avg)):
18     diff = abs(avg[i] - 1)
19     if diff < min_diff:
20         min_diff = diff
21         best_index = i
22 critical_L = Ls[best_index]

```

Multiple runs of the fine scanning algorithm yielded critical lengths ranging from 0.137 m to 0.142 m, with a typical value around 0.139-0.141 m. This narrow range of only 0.005 m variation demonstrates strong accuracy in the model when using a much higher number of initial fissions.

Initially attempted to use 1000 initial fissions, however there was still a lot of variation. Then used 10,000 but this caused a very long lag time in producing output so it was decided 5000 was the best value to use for the number of initial fissions.

5.4: 3D Fission Model Implementation

create a 3D more realistic model, the uranium is now modelled as a cube rather than a 1D bar. The `diffusion()` function introduces proper sampling from the diffusion distribution $s^2 \exp(-3s^2/2R^2)$, where the returned value is then multiplied by R to get the actual diffusion distance.

Spherical coordinates were then used to model each fission reaction. The azimuthal angle ϕ was distributed between 0 and 2π , while the co-latitude θ was sampled using $\cos \theta$ uniformly distributed between -1 and 1. This ensured that the model had neutrons emitted equally in all directions, as occurs in realistic fission processes.

```

1 def diffusion():
2     a = cos(2.0 * pi * random.random())
3     return sqrt(-0.667 * (log(random.random()) + log(random.random()) *
4         a * a))
5
6 L = 0.1
7 N_initial = 1000
8 secondary_count_3d = 0
9
10 for i in range(N_initial):
11     x0 = L * random.random()
12     y0 = L * random.random()
13     z0 = L * random.random()
14
15     for j in range(neutrons()):
16         s = diffusion() * R
17         phi = 2.0 * pi * random.random()
18         theta = acos(2.0 * random.random() - 1.0)
19
20         x_new = x0 + s * sin(theta) * cos(phi)
21         y_new = y0 + s * sin(theta) * sin(phi)
22         z_new = z0 + s * cos(theta)
23
24         if (0 <= x_new <= L and 0 <= y_new <= L and 0 <= z_new <= L):
25             secondary_count_3d += 1

```

Multiple 3D test runs with $L = 0.1$ m and $N = 1000$ were used to test the model and check it works well. This gave secondary fissions ranging from 559 to 628, giving an average range of 0.559 to 0.628.

5.5: 3D Critical Size and Mass Calculation

Here the critical values for the 3D model were found by testing multiple cube sizes and finding the corresponding average secondary fissions, and uranium mass. The density of 18.7 Mg/m^3 was used to convert volume to mass, with multiplication by 1000 to report results in kilograms.

```
1  lengths_3d = [0.10, 0.12, 0.14, 0.16, 0.18, 0.20]
2  density = 18.7
3
4  for L in lengths_3d:
5      secondary_total_3d = 0
6      for i in range(N_variable_3d):
7          x0 = L * random.random()
8          y0 = L * random.random()
9          z0 = L * random.random()
10
11         for j in range(neutrons()):
12             s = diffusion() * R
13             phi = 2.0 * pi * random.random()
14             theta = acos(2.0 * random.random() - 1.0)
15
16             x_new = x0 + s * sin(theta) * cos(phi)
17             y_new = y0 + s * sin(theta) * sin(phi)
18             z_new = z0 + s * cos(theta)
19
20             if (0 <= x_new <= L and 0 <= y_new <= L and 0 <= z_new <= L)
21                 :
22                 secondary_total_3d += 1
23
24     avg_per_fission_3d = secondary_total_3d / N_variable_3d
25     volume = L**3
26     mass = volume * density * 1000
```

The same fine scanning approach as the 1D case was then applied to find the precise critical values.

Multiple simulation runs showed consistent progression from approximately 0.59-0.61 at $L = 0.10 \text{ m}$ to 1.30-1.33 at $L = 0.20 \text{ m}$. Fine scanning between 0.14 and 0.16 m using 201 points gave a critical value for L consistently within the range 0.145-0.150 corresponding to critical mass values ranging from 56.891 kg to 59.401 kg.

This critical mass range for cubic uranium-235 aligns reasonably with established values for uranium spheres which is around 50 kg, acknowledging that spherical geometry is more efficient for neutron conservation. The relatively consistent critical values across multiple runs shows the reliability of the simulation for critical mass estimation.

The 1D model predicted critical length 0.137-0.142 m, while 3D gives 0.145-0.150 m, both agree closely, showing 1D can provide reasonable estimates but is not as precise as the 3D model.